

LAYER: 2_MesCore

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: `**/obj/**`, `**/bin/**`, `**/*.png`, `**/*.jpg`, `**/node_modules/**`, `**/*.Designer.cs`
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

```
Promatis.Net.MES.Configuration/Class1.cs
Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj
Promatis.Net.MES.Data/Configuration/TechnologicalOperationParameterBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs
Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs
Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj
Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs
Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs
Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs
Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj
Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationHierarchyEngine.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/TechnologicalOperationParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/TechnologicalParameterCalcMethodBase.cs
```

```

Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitBase.cs
Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs
Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/ITechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/TechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/ITechnologicalParameterService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/TechnologicalParameterService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs
Promatis.Net.MES.Service/UnitService/UnitBaseService.cs
Promatis.Net.MES.Tests/AssemblyInfo.cs
Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj
Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs
Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs
Promatis.Net.MES.Tests/UnitBase/TestUnit.cs
Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs
Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs
Promatis.Net.MES.UI/_Imports.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor.cs
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPageContext.cs
Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj
Promatis.Net.MES.UI/UiModule.cs
Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

```

<file path="Promatis.Net.MES.Configuration/Class1.cs">
namespace Promatis.Net.MES.Configuration
{
    public class Class1
    {
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
</Project>
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/
TechnologicalOperationParameterBaseConfiguration.cs">

```

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

public abstract class TechnologicalOperationParameterBaseConfiguration<T, TOperation, TParameter> :
    IEntityTypeConfiguration<T>
{
    where T : TechnologicalOperationParameterBase<TOperation, TParameter> // <--- A05D 5CD0CT< C0;CâAC08CR ?C
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : DomainObject, ITechnologicalParameter
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C0A CD;D0 ?D 5CD>D$2D 0D"5C08D0 4D41C'8D >C$0C08D0 ?C @C <CTBD >C" =C >C
        builder.HasIndex(x => new { x.OperationId, x.ParameterId })
            .IsUnique();
    }
}
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

/// <summary>
/// A0>C0DC,,3D4@C FC,,O CD;D0 1C 7D² AC$0Ct8D
/// </summary>
/// <typeparam name="T"></typeparam>
public abstract class TechnologicalOperationUnitBaseConfiguration<T, TOperation> :
    IEntityTypeConfiguration<T>
{
    where T : TechnologicalOperationUnitBase<TOperation> // <--- A05D 5CD0CT< D$8C0 >C05D 0Dd8C•
    where TOperation : DomainObject, ITechnologicalOperation
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C0A CD;D0 ?C @D² ?Cä;CT9 (A0@CT4CâBC$@C IC 5D" 4D41C'8D >C$0C08CR AC$0Ct5
        builder.HasIndex(x => new { x.OperationId, x.UnitId })
            .IsUnique();
    }
}
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.Data;

public class UnitBaseConfiguration : IEntityTypeConfiguration<UnitBase>
{
    public void Configure(EntityTypeBuilder<UnitBase> builder)
    {

```

```

        // A$:C'NDt0CT< D BD 0D$5C48Dâ E B 4C'0 C$ACT9 C,,5D 0D EC,,8 UnitBaseD
        builder.UseTptMappingStrategy();D
D
        // A00D BD >C":C ?Cä;CT9D
        builder.Property(x => x.Kind).IsRequired();D
        builder.Property(x => x.Type).IsRequired();D
    }D
}
</file>

<file path="Promatis.Net.MES.Data/DBContext/MesApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;D
using Microsoft.Extensions.Configuration;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
public class MesApplicationDbContext(D
    DbContextOptions options,D
    IConfiguration configuration,D
    IServiceProvider? serviceProvider = null) D
    : ApplicationDbContext(options, configuration, serviceProvider)D
{D
    public DbSet<UnitOfMeasurement> UnitOfMeasurements => Set<UnitOfMeasurement>();D
}
</file>

<file path="Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">D
D
    <PropertyGroup>D
        <TargetFramework>net10.0</TargetFramework>D
        <ImplicitUsings>enable</ImplicitUsings>D
        <Nullable>enable</Nullable>D
    </PropertyGroup>D
D
    <ItemGroup>D
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />D
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />D
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />D
    </ItemGroup>D
D
    <ItemGroup>D
        <ProjectReference Include="..\..\Core\Data\Promatis.Net.Data.csproj" />D
        <ProjectReference Include="..\Promatis.Net.MES.Domain\Promatis.Net.MES.Domain.csproj" />D
    </ItemGroup>D
D
</Project>
</file>

<file path="Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
/// <summary>D
/// B4=C,,2CT@D 0C'LC0KC' ?CT@CTEC$0D$GC,,: B #A , C¤>C0BD >C'8D CDäIC,,9 C0@C 2C,,;C 2C'>Cd5C0=CäAD$8 D$5DT=Cä
/// </summary>D
public class TechnologicalOperationHierarchyTrigger<T, TOLink, TPLink> : IBeforeSaveTrigger<T>D
    where T : TechnologicalOperationBase<T, TOLink, TPLink>D
    where TOLink : classD
    where TPLink : classD
{D
    public async Task HandleAsync(EntityCancelEventArgs<T> args)D
    {D
        T operation = args.Entity;D
D
        // ATAC'8 Cä?CT@C FC,,0 C¤>D =CT2C 0 – C0@Cä?D4AC¤0CT< C0@Cä2CT@C¤C0
        if (!operation.ParentId.HasValue || operation.ParentId == Guid.Empty) return;D
D
        // A,,7C$;CT:C 5CÂ 8Cr !B4 AB AD$0D$CD –4Æv b 8 Name D >CD8D$5C'LD :Cä9 Cä?CT@C FC,,8D
        var parentInfo = await args.Context.Set<T>().D
            .Where(x => x.Id == operation.ParentId)D

```

```

        .Select(x => new { x.IsLeaf, x.Name })
        .FirstOrDefaultAsync();
    }

    if (parentInfo != null)
    {
        // A,,ACô>C´LctCCT< CÔ0D, 4Cä<CT=CÔKC´ 4C$8Cd>C 4C´O Cô@Cä2CT@C=8 Cô@C 2C,,; C$;Cä6CT=CÔ>D BC,1
        // Aô5D 5CD0CT< D BC BD4A IsLeaf D >CD8D$5C´O, C$KD$0D"5CÔ=D´9 C,,7 C 0CtK.
        if (parentInfo.IsLeaf)
        {
            args.Cancel = true;
            args.ErrorMessage = $"Aô0D CD,,5CÔ8CR 8CT@C @DT8C, ÔU3  "CTECÔ>C´>C48Dt5D :C 0 Cä?CT@C FC,,O '{
                $"CÔ5 CÄ>Cd5D" 1D´BDÂ 2C´>Cd5CÔ0 C" w. &VçD-æfðäæ ÖWôrÂ BC : C=0C  >CÔ0

            return;
        }
    }
}

// AD>Cô>C´=C,,BCT;DÄ=C 0 Ct0D"8D$0 CÔ0 C,,7CÄ5CÔ5CÔ8CS 5D ;C, >Cô5D 0Dd8Dò AD$0CÔ>C$8D$ADò C,,AD$>CÄÂ
// Cô@Cä2CT@Dô5CÄ 2 ChangeTracker C,,;C, ABÂ =CTB C´8 D2 =CT5 CD>Dt5D =C,,E DÔ;CT<CT=D$>C-
if (operation.IsLeaf)
{
    bool hasChildrenInDb = await args.Context.Set<T>().
        .AnyAsync(x => x.ParentId == operation.Id);

    if (!TechnologicalOperationHierarchyEngine.CanChangeLeafStatus(hasChildrenInDb,
operation.IsLeaf))
    {
        args.Cancel = true;
        args.ErrorMessage = $"Aô0D CD,,5CÔ8CR FCT;CäAD$=CäAD$8 CD0CÔ=D´E: Aô5C$>Ct<Cä6CÔ> D 4CT;C BDÂ
            $"D$0C : C : C$=D4BD 8 CÔ5CR CCD5 D >CD5D 6C BD 0 CD@D43C,,5 D$5DT=Cä;Cä3C

    }
}
}
</file>

<file path="Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

/// <summary>
/// B$@C,,3C45D 4C´O Cô@Cä2CT@C=8 D ?CTFC,,DC,,GCÔKDR ?D 0C$8C² 8CT@C @DT8C, Væ-D& 6Rí
/// </summary>
public class UnitBaseHierarchyTrigger : IBeforeSaveTrigger<UnitBase>
{
    public async Task HandleAsync(EntityCancelEventArgs<UnitBase> args)
    {
        UnitBase unit = args.Entity;

        if (!unit.ParentId.HasValue || unit.ParentId == Guid.Empty) return;

        // Aô>C´CDt0CT< Kind C, æ ÖR @Cä4C,,BCT;Dò 4C´O C,,=DD>D <C BC,,2CÔ>C4> D >Cä1D"5CÔ8Dý
        var parentInfo = await args.Context.Set<UnitBase>().
            .Where(x => x.Id == unit.ParentId)
            .Select(x => new { x.Kind, x.Name })
            .FirstOrDefaultAsync();

        if (parentInfo != null && !UnitHierarchyEngine.IsHierarchyValid(parentInfo.Kind, unit.Kind))
        {
            args.Cancel = true;
            args.ErrorMessage = $"Aô0D CD,,5CÔ8CR 8CT@C @DT8C, ÔU3 C JCT:D" w.Væ-Bäæ ÖWôr ‡.Væ-Bä¶-æGò' " ½
                $"CÔ5 CÄ>Cd5D" 1D´BDÂ 2C´>Cd5CÔ 2 '{parentInfo.Name}' ({parentInfo.Kind}).";

        }
    }
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs">
using System.ComponentModel;

namespace Promatis.Net.MES.Domain.Interface;

/// <summary>

```

```

/// AÄ5D$>CDK D 0D GCTBC BCTE. C00D 0CÄ5D$@Cä2D
/// </summary>D
[Flags]D
public enum CalculationMethodD
{D
    [Description("A05 Ct0CD0C0"•Ý
    None = 0, // 00000000D
D
    [Description("AÄ0C0AC,,<C ;DÄ=Cä5 Ct=C GCT=C,,5 (MAX)")]D
    Max = 1,D
D
    [Description("AÄ8C08CÄ0C´LC0>CR 7C00Dt5C08CR „0"â'"•Ý
    Min = 2,D
D
    [Description("B @CT4C05CR 7C00Dt5C08CR „ dr'"•Ý
    Avg = 4,D
D
    [Description("A05D 2Cä5 Ct=C GCT=C,,5 (FIRST)")]D
    First = 8,D
D
    [Description("A0>D ;CT4C05CR 7C00Dt5C08CR „Ä 5B'"•Ý
    Last = 16,D
D
    [Description("A$ACR 7C00Dt5C08D0 „ ÄÄ'"•Ý
    All = 32D
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs">
using System.ComponentModel;D
D
namespace Promatis.Net.MES.Domain.Interface;D
D
public enum UnitKindD
{D
    [Description("B :C´0CDAC0D0 ;Cä3C,,AD$8C0")D
    Storage = UnitType.Storage | UnitType.Zone | UnitType.Rack | UnitType.Cell | UnitType.Crane,D
D
    [Description("A0@Cä8Ct2Cä4D BC$5C0=C 0 Ct>C00")D
    Production = UnitType.Workshop | UnitType.Section | UnitType.Line | UnitType.Workstation |
UnitType.MachineTool | UnitType.Table,D
D
    [Description("B$@C =D ?Cä@D$=D´9 D47CT;")]D
    Transport = UnitType.Vehicle | UnitType.Conveyor,D
D
    [Description("A0>CD@C 7CD5C´5C08CR"•Ý
    Department = UnitType.Workshop | UnitType.Section | UnitType.Other,D
D
    [Description("B 0C >Dt0D0 BCäGC0 / B0GCT9C0")D
    Position = UnitType.Cell | UnitType.OtherD
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs">
using System.ComponentModel;D
D
namespace Promatis.Net.MES.Domain.Interface;D
D
[Flags]D
public enum UnitTypeD
{D
    None = 0,D
D
    [Description("Bd5DR"•Ý
    Workshop = 1,D
D
    [Description("B4GC AD$>C¢"•Ý
    Section = 2,D
D
    [Description("A´8C08D0 0 Cä=C$5C"5D "•Ý
    Line = 4,D
D
    [Description("B 0C >Dt5CR <CTAD$>")]D
    Workstation = 8,D
D
    [Description("B :C´0CB"•Ý

```

```

    Storage = 16,0
}
[Description("At>C00 DT@C =CT=C,,0")]0
Zone = 32,0
}
[Description("B BCT;C´0Cb"•Ý
Rack = 64,0
}
[Description("B0GCT9C00 C 4D 5D 0 DT@C =CT=C,,0")]0
Cell = 128,0
}
[Description("A0@C = / A0>CDJCT<C08C0"•Ý
Crane = 256,0
}
[Description("B BC =Cä:")]0
MachineTool = 512,0
}
[Description("A$5D AD$0C0 0 !D$>C²"•Ý
Table = 1024,0
}
[Description("B$@C =D ?Cä@D$=Cä5 D @CT4D BC$>")]0
Vehicle = 2048,0
}
[Description("A 2D$>C0>CÄ=D´9 D$@C =D ?Cä@D$5D "•Ý
Conveyor = 4096,0
}
[Description("A0@CäGCT5")]0
Other = 81920
}
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs">
namespace Promatis.Net.MES.Domain.Interface;0
}
public interface ITechnologicalOperation0
{0
    0
}
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs">
namespace Promatis.Net.MES.Domain.Interface;0
}
public interface ITechnologicalParameter0
{0
    0
}
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">0
}
<PropertyGroup>0
    <TargetFramework>net10.0</TargetFramework>0
    <ImplicitUsings>enable</ImplicitUsings>0
    <Nullable>enable</Nullable>0
</PropertyGroup>0
}
<ItemGroup>0
    <ProjectReference Include="..\..\Core\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />0
</ItemGroup>0
}
</Project>
</file>

<file path="Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">0
}
<PropertyGroup>0
    <TargetFramework>net10.0</TargetFramework>0
    <ImplicitUsings>enable</ImplicitUsings>0
    <Nullable>enable</Nullable>0
</PropertyGroup>0
}
<ItemGroup>0
    <ProjectReference Include="..\..\Core\Domain\Promatis.Net.Domain.csproj" />0

```

```

    <ProjectReference Include="..\
\Promatis.Net.MES.Domain.Interface\Promatis.Net.MES.Domain.Interface.csproj" />
</ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// A 0Ct>C$0D0 BCTEC0>C'>C48Dt5D :C 0 Cä?CT@C FC,,0.0
/// A00D ;CT4D45D" GC,,AD$>CR =CR04Cd5C05D 8C¢ 4CT@CT2Câ !B4 ABÂ CC 8D 0D0 :Cä=DD;C,,:D$K CÄ0C0?C,,=C40,0
/// C, ?D 5CD>D BC 2C'0CTB D BD >C4> D$8C08Ct8D >C$0C0=D'9 C,,=D$5D DCT9D 4C'0 C 8Ct=CTA-C'>C48C¤8.0
/// </summary>
public abstract class TechnologicalOperationBase<T, TOLink, TPLink> : ReferenceTreeBase<T>, // <- A¤ A,, "A,, 'AT!A¤ AR B B A$ A
ITechnologicalOperation
where T : TechnologicalOperationBase<T, TOLink, TPLink>
where TOLink : class
where TPLink : class
{
    public bool IsLeaf { get; set; } = true;
    public virtual ICollection<TOLink> UnitLinks { get; set; } = new List<TOLink>();
    public virtual ICollection<TPLink> ParameterLinks { get; set; } = new List<TPLink>();
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
// A$0C'8CD0D$>D >C05D 0Dd8C, ,,>D BC 5D$AD0 1CT7 C,,7CÄ5C05C08C'•
public abstract class TechnologicalOperationBaseValidator<T> : ReferenceTreeBaseValidator<T>
where T : ReferenceTreeBase<T>, ITreeNode<T>, ITechnologicalOperation // <- A¤ A,, "A,, 'AT!A¤ AR B B A$ A
{
    protected TechnologicalOperationBaseValidator() : base()
    {
        // A 0Ct>C$KC' &VfW&Væ6T& 6Uf Æ-F F=" CCd5 C0@Cä2CT@C,,; Cä1D"8CR ?Cä;D0 ,, -BÂ æ ÖR•
        // At4CTADÂ <D² ?C,,HCT< C0@C 2C,,;C AD$@Cä3Cä 4C'0 D$5DT=Cä;Cä3C,,GCTAC¤8DR >C05D 0Dd8C•
        RuleFor(x => x.Code)
            .NotEmpty()
            .WithMessage("A¤>CB >C05D 0Dd8C, >C 0Ct0D$5C'5C0â"•
            .MaxLength(50)
            .WithMessage("A¤>CB >C05D 0Dd8C, =CR 4Cä;Cd5C0 ?D 5C$KD,,0D$L 50 D 8CÄ2Cä;Cä2.");
    }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/
TechnologicalOperationHierarchyEngine.cs">
using System.Reflection;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.MES.Domain;
public static class TechnologicalOperationHierarchyEngine
{
    /// <summary>
    /// A4;Cä1C ;DÄ=D'9 C,,AD$>Dt=C,,: C0@C 2CDK: C0@Cä2CT@D05D"Â 2C ;C,,4C0> C'8 C$;Cä6CT=C,,5 CD>Dt5D =CT9 Cä?C
    /// A00CÄ5D BC$> C ;Cä:C,,@D45D" ACä7CD0C08CR 2CTBCä: C$=D4BD 8 D$5D <C,,=C ;DÄ=D'E Cä?CT@C FC,,9 (IsLeaf =
    /// B$8C08Ct0Dd8D0 ?CT@CT2CT4CT=C =C GC,,AD$KC' :Cä=D$@C :D" 4CT@CT2C BÂ GD$> C0>C'=CäAD$LDâ CD BD 0C00
    /// </summary>
    /// <typeparam name="T">A¤>C0:D 5D$=D'9 D$8C0 BCTEC0>C'>C48Dt5D :Cä9 Cä?CT@C FC,,8</typeparam>
    /// <param name="parent">B >CD8D$5C'LD :C 0 Cä?CT@C FC,,0</param>
    /// <param name="childIsLeaf">A0@C,,7C00C¢ BCä3CäÂ 0C$;D05D$AD0 ;C, ACä7CD0C$0CT<D'9/C05D 5CÄ5D"0CT<D'9 D
    public static bool IsHierarchyValid<T>(T? parent, bool childIsLeaf)

```



```

    where T : class, ITreeNode<T>ð
{ð
    // 1. ATAC´8 D >CD8D$5C´O CÔ5D"Â <D² ACä7CD0CT< C¤>D =CT2Cä9 DÔ;CT<CT=D" r MD$> C$ACT3CD0 C$0C´8CD=Cä
    if (parent == null)ð
    {ð
        return true;ð
    }ð
ð
    // 2. Bt5D 5Cr @CTDC´5C¤AC,,N (C,,;C, :C AD" : C,,=D$5D DCT9D C, CTAC´8 C K Cä= C KC²´ 8Ct2C´5C¤0CT< C0@
    // B$0C¢ :C : T CÔ0D ;CT4D45D$ADò >D" FV6†æöÆöv-6 Ä÷ W& F-öä& 6RÂ AC$>C"AD$2Cä -4ÆV b BC < C40D 0CÔBC
    PropertyInfo? isLeafProperty = typeof(T).GetProperty("IsLeaf");ð
    if (isLeafProperty != null)ð
    {ð
        bool parentIsLeaf = (bool)isLeafProperty.GetValue(parent)!;ð
        if (parentIsLeaf)ð
        {ð
            return false; // B$5D <C,,=C ;DÄ=D´9 D47CT; CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT<ð
        }ð
    }ð
ð
    return true;ð
}ð
ð
    /// <summary>ð
    /// A0@Cä2CT@Dô5D"Â @C 7D 5D,,5CÔ> C´8 Cô5D 5C¤;DäGC BDÂ DC´0C2 -4ÆV b C D CD"5D BC$CDäICT9 Cä?CT@C FC,,8.D
    /// ATAC´8 D2 >Cô5D 0Dd8C, CCd5 DD8Ct8Dt5D :C, 5D BDÂ 4CäGCT@CÔ8CR 2CTBC¤8 C" At#, C0@CT2D 0D"0D$L CTQ C
    /// </summary>ð
    public static bool CanChangeLeafStatus(bool hasChildren, bool requestedIsLeaf)ð
    {ð
        if (hasChildren && requestedIsLeaf)ð
        {ð
            return false;ð
        }ð
    }ð
ð
    return true;ð
}ð
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// B 2Dô7D4ND"0Dò AD4ICÔ>D BDÂ ´ CÔ>C48CR0:Cä0<CÔ>C48CĚ² <CT6CDC D$5DT=Cä;Cä3C,,GCTAC¤8CÄ8 Cä?CT@C FC,,0CÄ8 C,
/// Aä?D 5CD5C´OCTB D$@CT1Cä2C =C,,0 C¢ 7C ?Cä;CÔ5CÔ8Dâ ?C @C <CTBD >C" 4C´O C¤>CÔ:D 5D$=Cä9 Cä?CT@C FC,,8.D
/// </summary>ð
public abstract class TechnologicalOperationParameterBase<TOperation, TParameter> : DomainObject,
ISoftDeletableð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : DomainObject, ITechnologicalParameterð
{ð
    public Guid OperationId { get; set; }ð
ð
    /// <summary>ð
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : C 0Ct>C$>C´ >Cô5D 0Dd8C,i
    /// </summary>ð
    public virtual TOperation Operation { get; set; } = null!;ð
ð
    public Guid ParameterId { get; set; }ð
ð
    /// <summary>ð
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : C 0Ct>C$>CÄC C00D 0CÄ5D$@D2i
    /// </summary>ð
    public virtual TParameter Parameter { get; set; } = null!;ð
ð
    /// <summary>ð
    /// A0@C,,7CÔ0C¢ BCä3CäÂ GD$> C00D 0CÄ5D$@ Cä1Dô7C BCT;CT= CD;Dò 7C ?Cä;CÔ5CÔ8Dò1
    /// </summary>ð
    public bool IsRequired { get; set; } = false;ð
ð
    /// <summary>ð

```

```

    /// AÔ>CÄ8CÔ0C´LCÔ>CR 7CÔ0Dt5CÔ8CR ?C @C <CTBD 0 CÔ> D4<Cä;Dt0CÔ8Däí
    /// </summary>ð
    public string? DefaultValue { get; set; }ð
ð
    /// <summary>ð
    /// AÄ8CÔ8CÄ0C´LCÔ> CD>CÔCD BC,,<Cä5 Ct=C GCT=C,,5.ð
    /// </summary>ð
    public double? MinValue { get; set; }ð
ð
    /// <summary>ð
    /// AÄ0CðAC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÔ0Dt5CÔ8CRí
    /// </summary>ð
    public double? MaxValue { get; set; }ð
ð
    public DateTime? DeletedAt { get; set; }ð
    public string? DeletedBy { get; set; }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public abstract class TechnologicalOperationParameterBaseValidator<T, TOperation, TParameter> :
DomainObjectValidator<T>ð
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>ð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : TechnologicalParameterBaseð
{ð
    protected TechnologicalOperationParameterBaseValidator()ð
    {ð
        RuleFor(x => x.OperationId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D >Cô5D 0Dd8C, >C 0ct0D$5C´5Còâ""½
ð
        RuleFor(x => x.ParameterId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1Dô7C BCT;CT=.");ð
ð
        // Að@CäAD Ô?Cä;CT2C 0 C$0C´8CD0Dd8Dó¢ C :D 8CÄ0C´LCÔ>CR 7CÔ0Dt5CÔ8CR =CR <Cä6CTB C KD$L CÄ5CÔLD,,5 C
        RuleFor(x => x.MaxValue)ð
            .GreaterThanOrEqualTo(x => x.MinValue)ð
            .When(x => x.MinValue.HasValue && x.MaxValue.HasValue)ð
            .WithMessage("AÄ0CðAC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÔ0Dt5CÔ8CR =CR <Cä6CTB C KD$L CÄ5CÔLD,,5 CÄ8CÔ8C
    }ð
    }
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// A 1D BD 0CðBCÔ0Dò 1C 7C 4C´O D 2Dô7C, ,,@C AD,,8D OCT<C 0>ð
/// </summary>ð
public abstract class TechnologicalOperationUnitBase<TOperation> : DomainObject, ISoftDeletableð
    where TOperation : DomainObject, ITechnologicalOperationð
{ð
    public Guid OperationId { get; set; }ð
ð
    /// <summary>ð
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : Cä?CT@C FC,,8 (C CCD5D" 7C :D KD$> Cð>CÔ:D 5D$=D´< D$8Cô>CÄ 2 MDM)
    /// </summary>ð
    public virtual TOperation Operation { get; set; } = null!;ð
ð
    public Guid UnitId { get; set; }ð
ð
    /// <summary>ð
    /// Aô@Dô<C 0 D AD´;Cð0 CÔ0 C 0ct>C$KC´ :C´0D A Cä1Cä@D44Cä2C =C,,0. ð

```

```

    /// EF Core C 2D$>CÄ0D$8Dt5D :C, AC$0Cd5D" MD$> D BC 1C´8Dd5C´ >C >D CCD>C$0C08Dò ?Câ ACä3C´0D,,5C08Dô<.>
    /// </summary>ð
    public virtual UnitBase Unit { get; set; } = null!;ð
ð
    public int Priority { get; set; } = 1;ð
ð
    public DateTime? DeletedAt { get; set; }ð
    public string? DeletedBy { get; set; }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/
TechnologicalOperationUnitBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// A 0Ct>C$KC´ 2C ;C,,4C BCä@ D 2Dô7C•
/// </summary>ð
public abstract class TechnologicalOperationUnitBaseValidator<T, TOperation> :
DomainObjectValidator<T>ð
    where T : TechnologicalOperationUnitBase<TOperation>ð
    where TOperation : DomainObject, ITechnologicalOperationð
{ð
    protected TechnologicalOperationUnitBaseValidator()ð
    {ð
        RuleFor(x => x.OperationId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D >Cô5D 0Dd8C, >C 0Ct0D$5C´5C0â""½
ð
        RuleFor(x => x.UnitId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D >C >D CCD>C$0C08Dò >C 0Ct0D$5C´5C0â""½
ð
        RuleFor(x => x.Priority)ð
            .InclusiveBetween(1, 10)ð
            .WithMessage("Aô@C,,>D 8D$5D" >C >D CCD>C$0C08Dò 4Cä;Cd5Cò 1D´BDÂ 2 CD8C ?C 7Cä=CR >D" 4Cä ä""
    }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// A 0Ct>C$KC´ BCTECô>C´>C48Dt5D :C,,9 Cò0D 0CÄ5D$@ (D ?D 0C$>Dt=C,,: DT0D 0CðBCT@C,,AD$8Cð ?D >Dd5D ACä2).ð
/// </summary>ð
public abstract class TechnologicalParameterBase : ReferenceBase, ITechnologicalParameterð
{ð
    /// <summary>ð
    /// A$=CTHC08C´ :C´Ndr =C ACô@C 2CäGC08Cð 5CD8C08Db 8Ct<CT@CT=C,,0.ð
    /// </summary>ð
    public Guid? UnitOfMeasurementId { get; set; }ð
ð
    public virtual UnitOfMeasurement? UnitOfMeasurement { get; set; }ð
ð
    /// <summary>ð
    /// B$8Cò 4C =C0KDR ?C @C <CTBD 0 (C00C0@C,,<CT@: Bt8D ;CâÂ !D$@Cä:C Â D4;CT2Cä´í
    /// </summary>ð
    public string DataType { get; set; } = "Numeric";ð
ð
    /// <summary>ð
    /// B 0Ct@CTHCT=C0KCR <CTBCä4D² @C AD GE BC
    /// </summary>ð
    public CalculationMethod AllowedMethods { get; set; } = CalculationMethod.None;ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs">
using FluentValidation;ð

```

```

using Promatis.Net.Domain;
namespace Promatis.Net.MES.Domain;
public abstract class TechnologicalParameterBaseValidator<T> : ReferenceBaseValidator<T>
    where T : TechnologicalParameterBase
{
    private static readonly string[] AllowedDataTypes = ["Numeric", "String", "Boolean",
"DateTime"];
    protected TechnologicalParameterBaseValidator() : base()
    {
        // 1. A$0C'8CD0Dd8D0 5CD8C08DdK C,,7CÄ5D 5C08D0 „4CäAD$CC0=C Â BC : C0C¢ B r MD$> TechnologicalParame
RuleFor(x => x.UnitOfMeasurement)
        .NotNull()
        .WithMessage("AT4C,,=C,,FC 8Ct<CT@CT=C,,0 C05 CÄ>Cd5D" 1D'BDÄ çVÆÄâ""½
    }
    // 2. A$0C'8CD0Dd8D0 BC,,?C 4C =C0KD]
RuleFor(x => x.DataType)
        .NotEmpty()
        .WithMessage("B$8C0 4C =C0KDR ?C @C <CTBD 0 Cä1D07C BCT;CT=.")
        .Must(type => AllowedDataTypes.Contains(type))
        .WithMessage($"A05CD>C0CD BC,,<D'9 D$8C0 4C =C0KDRâ C 7D 5D,,5C0=D'5 Ct=C GCT=C,,0: {string.Join(",
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
public abstract class TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter> :
DomainObject, ISoftDeletable
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    public Guid UnitId { get; set; }
    /// <summary>
    /// Bd5DT>C$0D0 5CD8C08Dd0
    /// </summary>
    public virtual TUnit Unit { get; set; } = null!;
    public Guid TechnologicalOperationId { get; set; }
    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC0D0 >C05D 0Dd8Dý
    /// </summary>
    public virtual TOperation TechnologicalOperation { get; set; } = null!;
    public Guid TechnologicalParameterId { get; set; }
    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC08C' ?C @C <CTBD
    /// </summary>
    public virtual TParameter TechnologicalParameter { get; set; } = null!;
    /// <summary>
    /// AÄ5D$>CB @C ADt5D$0
    /// </summary>
    public CalculationMethod CalculationMethod { get; set; } = CalculationMethod.None;
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;

```

```

using Promatis.Net.MES.Domain.Interface;
{
namespace Promatis.Net.MES.Domain;
{
public abstract class TechnologicalParameterCalcMethodBaseValidator<T, TUnit, TOperation,
TParameter> : DomainObjectValidator<T>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    protected TechnologicalParameterCalcMethodBaseValidator()
    {
        RuleFor(x => x.UnitId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½

        RuleFor(x => x.TechnologicalOperationId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä9 Cä?CT@C FC,,8 Cä1Dô7C BCT;CT=

        RuleFor(x => x.TechnologicalParameterId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä3Câ ?C @C <CTBD 0 Cä1Dô7C BCT;

        // Aô@Cä2CT@Dô5CÄÂ GD$> CÄ5D$>CB @C ADt5D$0 C KC² OC$=Cä 2D´1D 0C0 ?Cä;DÄ7Cä2C BCT;CT<
        RuleFor(x => x.CalculationMethod)
            .NotEqual(CalculationMethod.None)
            .WithMessage("A05Cä1DT>CD8CÄ> C$KC @C BDÄ :Cä@D 5C=BC0KC´ <CTBCä4 D 0D GCTBC ?C @C <CTBD >C"â""½
    }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs">
using Promatis.Net.Domain;
{
namespace Promatis.Net.MES.Domain;
{
public abstract class TechnologicalParameterValueBase<TUnit, TParameter> : DomainObject
    where TUnit : UnitBase
    where TParameter : TechnologicalParameterBase
{
    public Guid UnitId { get; set; }

    /// <summary>
    /// Bd5DT>C$0D0 5CD8C08Dd0 (C,,AD$>Dt=C,,: CD0C0=D´E)
    /// </summary>
    public virtual TUnit Unit { get; set; } = null!;

    public Guid TechnologicalParameterId { get; set; }

    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC=8C´ ?C @C <CTBD
    /// </summary>
    public virtual TParameter TechnologicalParameter { get; set; } = null!;

    /// <summary>
    /// B KD >CR ,3D 0Ct=Cä5") Ct=C GCT=C,,5, C05D 5CD0C0=Cä5 CäB SCADA/IoT C,,;C, >C05D 0D$>D 00
    /// </summary>
    public string Value { get; set; } = string.Empty;

    /// <summary>
    /// B$>Dt=Cä5 C$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 (Time-Series CÄ5D$:C •
    /// </summary>
    public DateTime Date { get; set; } = DateTime.UtcNow;
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/
TechnologicalParameterValueBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
{
namespace Promatis.Net.MES.Domain;
{
public abstract class TechnologicalParameterValueBaseValidator<T, TUnit, TParameter> :
DomainObjectValidator<T>
    where T : TechnologicalParameterValueBase<TUnit, TParameter>
    where TUnit : UnitBase

```

```

where TParameter : TechnologicalParameterBase
{
    protected TechnologicalParameterValueBaseValidator()
    {
        RuleFor(x => x.UnitId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½

        RuleFor(x => x.TechnologicalParameterId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä3Câ ?C @C <CTBD 0 Cä1D07C BCT;

        RuleFor(x => x.Value)
            .NotNull().WithMessage("At=C GCT=C,,5 C00D 0CÄ5D$@C =CR <Cä6CTB C KD$L null.")
            // B 0Ct@CTHC 5CÄ ?D4AD$CDâ AD$@Cä:D2Â BC : C=0C¢ 4C´0 String-C00D 0CÄ5D$@Cä2 D0BCâ <Cä6CTB C KD$
            // C0> D41C,,@C 5CÄ AC´CDt0C"=D´5 C0@Cä1CT;D² ?Câ :D 0D0<D
            .Must(val => val == null || val.Trim() == val)
            .WithMessage("At=C GCT=C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C´8 Ct0C=0C0GC,,2C BDÄAD0 ?D >C 5C´0CÄ8."

        RuleFor(x => x.Date)
            .Must(date => date <= DateTime.UtcNow.AddSeconds(5))
            .WithMessage("A$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 C05 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD"5CÄâ""½
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// A 1D BD 0C=BC00D0 1C 7C 4C´0 C$ACTE Cä@C40C08Ct0Dd8Cä=C0KDR 8 C0@Cä8Ct2Cä4D BC$5C0=D´E CT4C,,=C,,F.D
/// </summary>
public abstract class UnitBase : ReferenceTreeBase<UnitBase>, IAudit
{
    public UnitKind Kind { get; init; }
    public required UnitType Type { get; set; }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// A 1D BD 0C=BC0KC´ 2C ;C,,4C BCä@ CD;D0 1C 7Cä2Cä3Câ :C´0D AC Væ-D& 6R1
/// </summary>
/// <typeparam name="T">B$8C0Â =C AC´5CDCCT<D´9 CäB UnitBase.</typeparam>
public abstract class UnitBaseValidator<T> : ReferenceBaseValidator<T>
    where T : UnitBase
{
    protected UnitBaseValidator() : base()
    {
        // A0@Cä2CT@C=0, DtBCâ 2 Type C05 C0@C,,HC´> C0CD BCä5/CD5DD>C´BC0>CR 7C00Dt5C08CR DC´0C4>C-
        RuleFor(x => x.Type).NotEmpty();

        // A0@Cä2CT@C=0 D >CäBC$5D$AD$2C,,0 A=0D$5C4>D 8C, ,,¶-æB´ 8 B$8C00 (Type) C00 CäAC0>C$5 C 8D$>C$KDR <C
        RuleFor(x => x)
            .Must(u => ((int)u.Kind & (int)u.Type) != 0)
            .WithMessage("B$8C0 =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs">
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
public static class UnitHierarchyEngine
{
    /// <summary>

```

```

/// A4;Cä1C ;DÄ=D´9 C,,AD$>Dt=C,,: Cð@C 2CDK: Cð@Cä2CT@Dð5D"Â 2C ;C,,4C0> C´8 C$;Cä6CT=C,,5 D 5C 5C0:C 2 D >
/// A,,AC0>C´LCtCCTBD 0 C00CÄ5D BC$> Cð0C¢ 2 B #A -D$@C,,3C45D 0DRÄ BC : C, 2 UI-C$0C´8CD0Dd8C,i
/// </summary>ð
public static bool IsHierarchyValid(UnitKind parentKind, UnitKind childKind)ð
{ð
    // 1. B$5D <C,,=C ;DÄ=D´9 D47CT; Position C08C¤>C44C =CR <Cä6CTB C KD$L D >CD8D$5C´5Cí
    if (parentKind == UnitKind.Position) return false;ð

    // 2. AD5C00D BC <CT=D" <Cä6CTB D >CD5D 6C BDÂ 2D Q, C¤@Cä<CR ÷6-F-öí
    if (parentKind == UnitKind.Department) return childKind != UnitKind.Position;ð

    // 3. B ?CTFC,,0C´8Ct8D >C$0C0=D´5 Ct>C0K C,,7Cä;C,,@Cä2C =D² 4D CC2 >D" 4D CC40, C0> CÄ>C4CD" ACä4CT@Cd
    if (parentKind is UnitKind.Production or UnitKind.Transport or UnitKind.Storage)ð
    {ð
        return childKind == parentKind || childKind == UnitKind.Position;ð
    }ð

    return true;ð
}ð
}ð
/// <summary>ð
/// A$>Ct2D 0D´0CTB D ?C,,ACä: Aä Aä B A0 B´% D$8C0>C" ...Væ-EG- R´ 4C´0 C$KC00CD0DäICT3Cä AC08D :C 2 UI,ð
/// CäAC0>C$KC$0D0ADÄ =C C BCT3Cä@C,,8 (UnitKind) D >CD8D$5C´LD :Cä3Cä Cct;C Ä 2 C¤>D$>D KC´ <D² ACT9Dt0
/// </summary>ð
public static IEnumerable<UnitType> GetAllowedChildTypes(UnitKind parentKind)ð
{ð
    var allTypes = (UnitType[])Enum.GetValues(typeof(UnitType));ð
    var allowedTypes = new List<UnitType>();ð

    foreach (UnitType type in allTypes)ð
    {ð
        if (type == UnitType.None) continue;ð

        // 1. B =C GC ;C =C ECä4C,,<, C¢ :C :Cä<D2 ?CäBCT=Dd8C ;DÄ=Cä<D2 ¶-æB >D$=CäAC,,BD 0 D0BCäB D$8C0í
        // B :C =C,,@D45CÄ 2D 5 C¤0D$5C4>D 8C,Ä GD$>C K C0>C00D$L, C" :C :D4N CÄ0D :D2 ?Cä?C 4C 5D" MD$>D"
        foreach (UnitKind potentialChildKind in (UnitKind[])Enum.GetValues(typeof(UnitKind)))ð
        {ð
            // ATAC´8 D$8C0 ?Cä1C,,BCä2Cä 2DT>CD8D" 2 D0BD2 :C BCT3Cä@C,,N0
            if (((int)potentialChildKind & (int)type) != 0)ð
            {ð
                // A, 5D ;C, ?D 0C$8C´0 C,,5D 0D EC,,8 B At AT(A .B" 2C´>Cd8D$L D0BD2 :C BCT3Cä@C,,N C" BCT
                if (IsHierarchyValid(parentKind, potentialChildKind))ð
                {ð
                    if (!allowedTypes.Contains(type))ð
                    {ð
                        allowedTypes.Add(type);ð
                    }ð
                }ð
            }ð
        }ð
    }ð

    return allowedTypes;ð
}ð
}ð
</file>

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs">
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// AD>CÄ5C0=C 0 D CD"=CäAD$L CT4C,,=C,,FD² 8Ct<CT@CT=C,,0 (A0!A,´ 4C´0 D$5DT=Cä;Cä3C,,GCTAC¤8DR ?C @C <CTBD >C"
/// </summary>ð
public class UnitOfMeasurement : ReferenceBaseð
{ð
    // B 2Cä9D BC$0 Id, Code, Name, Description C00D$8C$=Cä CC00D ;CT4Cä2C =D² >D" &VfW&Væ6T& 6Rí
    // Code - C¤@C BC¤>CR >C >Ct=C GCT=C,,5 (C00C0@C,,<CT@, "°C", "CÄ<", "Cä1/CÄ8C0"´í
    // Name - C0>C´=Cä5 C00C,,<CT=Cä2C =C,,5 (C00C0@C,,<CT@, "A4@C 4D4A Bd5C´LD 8Dò"Â $ C,,;C´8CÄ5D$@").ð
}ð
</file>

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð

```

```

Ð
namespace Promatis.Net.MES.Domain;Ð
Ð
public class UnitOfMeasurementValidator : ReferenceBaseValidator<UnitOfMeasurement>Ð
{Ð
    // A05D 5Cä?D 5CD5C´OCT< C 0Ct>C$CDâ @CT3D4;Dô@C¤C CÔ0 D 0D HC,,@CT=CÔCDâ „@D4A/C´0D"Â FC,,DD K, C0@Cä1CT;D
    protected override string CodeRegexPattern => @"^[\p{L}\p{N}_\.\°\/*\-\s\%]*$";Ð
    Ð
    protected override string CodeValidationMessage => "A¤@C BC¤>CR >C >Ct=C GCT=C,,5 CÄ>Cd5D" ACä4CT@Cd0D$L C
    Ð
    public UnitOfMeasurementValidator() : base()Ð
    {Ð
        // Aô@C 2C,,;Cä æ÷DV× G´ 8 Matches D46CR >D$@C 1CäBC ND" 2 C 0Ct>C$>CÂ :C´0D ACR ?Câ =C HCT<D2 =Cä2Cä<
    Ð
        RuleFor(x => x.Code)Ð
            .NotEmpty().WithMessage("A¤@C BC¤>CR >C >Ct=C GCT=C,,5 Cä1D07C BCT;DÄ=Cä 4C´0 Ct0C0>C´=CT=C,,0.")Ð
            .MaxLength(15).WithMessage("A¤@C BC¤>CR >C >Ct=C GCT=C,,5 C05 CD>C´6C0> C0@CT2D´HC BDÄ R AC,,<
    Ð
        RuleFor(x => x.Name)Ð
            .MaxLength(100).WithMessage("Aô>C´=Cä5 C00C,,<CT=Cä2C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0 C05 CD>C´
    }Ð
    }
</file>

<file path="Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
    Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
    Ð
    <ItemGroup>Ð
        <ProjectReference Include="..\..\Core\Service\Promatis.Net.Service.csproj" />Ð
        <ProjectReference Include="..\Promatis.Net.MES.Data\Promatis.Net.MES.Data.csproj" />Ð
    </ItemGroup>Ð
    Ð
</Project>
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public interface ITechnologicalOperationParameterService<T, TOperation, TParameter> :
IBaseService<T, Guid>Ð
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>Ð
    where TOperation : DomainObject, ITechnologicalOperationÐ
    where TParameter : TechnologicalParameterBaseÐ
{Ð
    /// <summary>Ð
    /// Aô>C´CDt8D$L C$ACR ?C @C <CTBD K, C0@C,,2D07C =C0KCR : C¤>C0:D 5D$=Cä9 Cä?CT@C FC,,8Ð
    /// </summary>Ð
    Task<List<T>> GetByOperationIdAsync(Guid operationId);Ð
    Ð
    /// <summary>Ð
    /// Aô>C´CDt8D$L D$>C´LC¤> Cä1D07C BCT;DÄ=D´5 C00D 0CÄ5D$@D² 4C´0 Cä?CT@C FC,,8Ð
    /// </summary>Ð
    Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId);Ð
    }
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.EntityFrameworkCore.Internal;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð

```



```

Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public abstract class TechnologicalOperationParameterService<T, TOperation, TParameter, TContext>(Ð
    IDbContextFactory<TContext> contextFactory)Ð
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalOperationParameterService<T,
TOperation, TParameter>Ð
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>Ð
    where TOperation : DomainObject, ITechnologicalOperationÐ
    where TParameter : TechnologicalParameterBaseÐ
    where TContext : DbContextÐ
{Ð
    public async Task<List<T>> GetByOperationIdAsync(Guid operationId)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
    Ð
        return await context.Set<T>().Ð
            .Where(x => x.OperationId == operationId)Ð
            .Include(x => x.Parameter) // B @C 7D2 ?Cä4D$0C48C$0CT< D ?D 0C$>Dt=C,,: C00D 0CÄ5D$@Cä2 CD;D0 T•
            .AsNoTracking()Ð
            .ToListAsync();Ð
    }Ð
}
Ð
public async Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId)Ð
{Ð
    await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
    Ð
    return await context.Set<T>().Ð
        .Where(x => x.OperationId == operationId && x.IsRequired)Ð
        .Include(x => x.Parameter)Ð
        .AsNoTracking()Ð
        .ToListAsync();Ð
    }Ð
}
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
ITechnologicalOperationService.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public interface ITechnologicalOperationService<T, TLink> : ITreeService<T>Ð
    where T : ReferenceTreeBase<T>, ITechnologicalOperation,
Promatis.Net.Domain.Interface.IDomainObjectHasKey<Guid>Ð
    where TLink : classÐ
{Ð
    /// <summary>Ð
    /// A0>C`CDT8D$L D ?C,,ACä: D 0Ct@CTHCT=C0>C4> Cä1Cä@D44Cä2C =C,,0 (Dä=C,,BCä2) CD;D0 :Cä=C@CTBC0>C' BCTEC0
    /// </summary>Ð
    Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId);Ð
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
TechnologicalOperationService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public abstract class TechnologicalOperationService<T, TLink, TContext>(DbContextFactory<TContext>
contextFactory)Ð
    : ReferenceTreeService<T, TContext>(contextFactory), ITechnologicalOperationService<T, TLink>Ð
    where T : ReferenceTreeBase<T>, ITechnologicalOperation, IDomainObjectHasKey<Guid>, new()Ð
    where TLink : TechnologicalOperationUnitBase<T>Ð
    where TContext : DbContextÐ
{Ð
    public async Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId)Ð

```

```

    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
    }

    // LINQ-Ct0C0@CäA CäBD 0C 0D$KC$0CTB C,,4CT0C'LC0>:D
    // A ;C 3Cä4C @D0 BCä<D2Ä GD$> C" FV6†æöÆöv-6 Ä÷ W& F-öäVæ-D& 6R AC$>C"AD$2Cä Væ-B
    // C,,<CT5D" AD$@Cä3C,,9 D$8C0 Væ-D& 6RÄ Tb 6÷&R ?CäAD$@Cä8D" GC,,AD$KC' 5 Ä0ä0"ä 0C$BCä<C BC,,GCTACä8.
    return await context.Set<TLink>()
        .Where(x => x.OperationId == operationId)
        .Select(x => x.Unit)
        .AsNoTracking()
        .ToListAsync();
    }
}

/// <summary>
/// AäAD$0C$;D05CÄ <CTBCä4 C 1D BD 0CäBC0KCÄ =C BCT:D4ICT< D4@Cä2C05. Aä>C0:D 5D$=D'9 C0@C,,:C'0CD=Cä9 D
/// D$5DT?D >Dd5D ACä2 D 0CÄ ?CT@CT>C0@CT4CT;C,,B CT3Cä 4C'0 D >Ct4C =C,,0 D 2Cä8DR ?Cä;C,,<Cä@DD=D'E Cä?CT@
/// </summary>
public abstract override Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public interface ITechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter> :
IBaseService<T, Guid>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    /// <summary>
    /// A00C"BC, :Cä=Cä@CTBC0CDä 8C0AD$@D4:Dd8Dä @C ADt5D$0 CD;D0 BD >C":Cf C >D CCD>C$0C08CR 2 C05D 0Dd8D
    /// </summary>
    Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId);
}

/// <summary>
/// A0>C'CDt8D$L C$ACR <CTBCä4D2 @C ADt5D$0 CD;D0 :Cä=Cä@CTBC0>C' >C05D 0Dd8C, =C >C >D CCD>C$0C08C•
/// </summary>
Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Internal;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public abstract class TechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter,
TContext>()
    IDbContextFactory<TContext> contextFactory)
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalParameterCalcMethodService<T,
TUnit, TOperation, TParameter>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()

```

```

        .FirstOrDefaultAsync(x => x.UnitId == unitId &&
                                x.TechnologicalOperationId == operationId &&
                                x.TechnologicalParameterId == parameterId);
    }
}
public async Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();

    return await context.Set<T>()
        .Where(x => x.TechnologicalOperationId == operationId && x.UnitId == unitId)
        .Include(x => x.TechnologicalParameter) // A0>CD3D CCd0CT< D 2D07C =C0KC' ?C @C <CTBD
        .AsNoTracking()
        .ToListAsync();
}
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
ITechnologicalParameterService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalParameterService<T> : IReferenceService<T>
    where T : TechnologicalParameterBase
{
    /// <summary>
    /// A0>C'CDt8D$ L C00D 0CÄ5D$@D²Ä >D$DC,,;DÄBD >C$0C0=D'5 C0> D$8C0C CD0C0=D'E (Numeric, String, Boolean, D
    /// </summary>
    Task<List<T>> GetByDataTypeAsync(string dataType);
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
TechnologicalParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Internal;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalParameterService<T, TContext>(IDbContextFactory<TContext>
contextFactory)
    : ReferenceService<T, TContext>(contextFactory), ITechnologicalParameterService<T>
    where T : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<List<T>> GetByDataTypeAsync(string dataType)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.DataType == dataType)
            .AsNoTracking()
            .ToListAsync();
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
/// <summary>
/// A0>C0BD 0C0B D 5D 2C,,AC0>C4> D ;Cä0 CD;D0 CC0@C 2C'5C08D0 AC0@C 2CäGC08C0>CÄ 5CD8C08Db 8Ct<CT@CT=C,,0.D
/// </summary>
public interface IUnitOfMeasurementService : IReferenceService<UnitOfMeasurement>
{
}
</file>

```

```

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

/// <summary>
/// B 5D 2C,,A D4?D 0C$;CT=C,,O CÔ>D <C BC,,2CÔ>-D ?D 0C$>Dt=Cä9 C,,=DD>D <C FC,,5C' 5CD8CÔ8Db 8Ct<CT@CT=C,,0.D
/// </summary>
public class UnitOfMeasurementService(IDbContextFactory<MesApplicationDbContext> contextFactory)
    : ReferenceService<UnitOfMeasurement, MesApplicationDbContext>(contextFactory),
IUnitOfMeasurementService
{
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public interface IUnitBaseService<TContext> : IReferenceTreeService<UnitBase, TContext> // <- A,,!Aô A A' Aô
D$8Cô0 C" @Cä4C,,BCT;DÄAC=8C' 8CÔBCT@DD5C"A!D
    where TContext : DbContext
{
    Task<List<UnitBase>> GetByKindAsync(UnitKind kind);
    Task<List<UnitBase>> GetByTypeAsync(UnitType type);
    Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type);
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/UnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public abstract class UnitBaseService<TContext>(IDbContextFactory<TContext> contextFactory)
    : ReferenceTreeService<UnitBase, TContext>(contextFactory), IUnitBaseService<TContext>
    where TContext : DbContext
{
    // =====
    // Aô A "BD B AT Aô+A' A !B$ A B$ B' BT#A$ ,, AT A' At#AT"B / Aô B Aä Aô Ad )D
    // =====

    /// <summary>
    /// A>CÔBD 0C=B DD0C @C,,:C,â !C <C @CT0C'8Ct0Dd8Dò 1D44CTB CÔ0Cô8D 0Cô0 CÔ0 D ;Cä9 CÔ8Cd5 D
    /// C" :Cä=C@CTBCÔ>CÂ ACT@C$8D 5, C=D$>D KC' 2C,,4C,,B CÔ>C'8CÄ>D DCÔKCR :C'0D AD² ,,FW 'FÖVçEVæ-B 8 CD@.
    /// </summary>
    public abstract override Task<UnitBase> CreateChildTemplateAsync(UnitBase parent);

    // =====
    // Bt B "B' AÄ B$ AD+ A$+A B A, A Aô+BR ,,!C$>C"AD$2C ¶-æB 8 Type D$5Cô5D L C$8CD=D² 8CD5C ;DÄ=Cä•
    // =====

    public async Task<List<UnitBase>> GetByKindAsync(UnitKind kind)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<UnitBase>()
            .AsNoTracking()
            .Where(x => x.Kind == kind)
            .ToListAsync();
    }

    public async Task<List<UnitBase>> GetByTypeAsync(UnitType type)

```

```

    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        // A„ACô>C`LCtCCT< Cô>C 8D$>C$>CR $ ", D$0C¢ :C : UnitType – DôBCä `fÆ w5Y
        return await context.Set<UnitBase>().
            .AsNoTracking().
            .Where(x => (x.Type & type) != 0).
            .ToListAsync();
    }
}

public async Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();
    return await context.Set<UnitBase>().
        .AsNoTracking().
        .Where(x => x.Kind == kind && (x.Type & type) != 0).
        .ToListAsync();
}
}
</file>

<file path="Promatis.Net.MES.Tests/AssemblyInfo.cs">
// AäBC¢;DäGC 5D" ?C @C ;C`5C`LCô>CR 2D`?Cä;Cô5Cô8CR BCTAD$>C" 2 DôBCä9 D 1Cä@C¢5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        <IsPackable>false</IsPackable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="coverlet.collector" Version="10.0.0">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
        </PackageReference>
        <PackageReference Include="FluentAssertions" Version="8.10.0" />
        <PackageReference Include="FluentValidation" Version="12.1.1" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
        <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
        <PackageReference Include="Moq" Version="4.20.72" />
        <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
        </PackageReference>
        <PackageReference Include="xunit.v3" Version="3.2.2" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.Mes.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
    <ItemGroup>
        <Using Include="Xunit" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs">
using FluentAssertions;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.Extensions.Configuration;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.MES.Service;

```

```

namespace Promatis.Net.MES.Tests;
{
    public class UnitServiceStandaloneTests
    {
        // 1. A„ACô>C´LCtCCT< C$5Ct4CR FW7DF$6ôçFW‡M
        private readonly IDbContextFactory<TestDbContext> _factory;
        private readonly DbContextOptions<TestDbContext> _options;

        public class TestUnit : Domain.UnitBase { }

        /// <summary>
        /// B$5D BCâ2D´9 D 5D 2C„A CD;Dò 2CT@C„DC„:C FC„8 C 8Ct=CTA-C$KC >D >Cç Væ-D& 6R 2 CÄ>CDCC´LCÔKDR BCTAD$0
        /// A„!Aô A A´ Aô : B 8C4=C BD4@C ?D 8C$5CD5C00 Cç AD$0C04C @D$C UnitBaseService D >CD=C„< generic-C00
        /// </summary>
        public class TestUnitService(IDbContextFactory<TestDbContext> factory)
            : UnitBaseService<TestDbContext>(factory)
        {
            /// <summary>
            /// Aô@CâAD$5C"HC O Ct0C4;D4HC=0 DD0C @C„:C, ?Cä;C„<Câ@DD8Ct<C 4C´O D4ACô5D„=Câ9 C= >CÄ?C„;DôFC„8 C 1
            /// </summary>
            public override Task<Domain.UnitBase> CreateChildTemplateAsync(Domain.UnitBase parent)
            {
                // A„!Aô A A´ Aô : A 5Ct>C00D =Câ5 CD8C00CÄ8Dt5D :Câ5 D >Ct4C =C„5 C„=D BC =D 0 C00D$8C$=Câ3Câ B
                var childMock = (Domain.UnitBase)Activator.CreateInstance(parent.GetType());

                childMock.ParentId = parent.Id;
                childMock.Parent = null;

                return Task.FromResult(childMock);
            }
        }

        public UnitServiceStandaloneTests()
        {
            // 2. Aä?Dd8C, 4Cä;Cd=D² 1D´BDÂ 4C´O TestDbContext
            _options = new DbContextOptionsBuilder<TestDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .Options;

            // 3. AÄ>C=0CT< DD0C @C„:D2 8CÄ5C0=Câ 4C´O TestDbContext
            var factoryMock = new Mock<IDbContextFactory<TestDbContext>>();

            factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
                .ReturnsAsync(() => new TestDbContext(_options)); // ReturnsAsync D4?D >D´0CTB Task.FromResult

            _factory = factoryMock.Object;

            // 4. A= >C0BCT:D B D$>Cd5 CD>C´6CT= C0@C„=C„<C BDÂ ?D 0C$8C´LCÔKCR >C0FC„8D
            public class TestDbContext(DbContextOptions<TestDbContext> options)
                : ApplicationDbContext(options, new ConfigurationBuilder().Build())
            {
                protected override void OnModelCreating(ModelBuilder modelBuilder)
                {
                    base.OnModelCreating(modelBuilder);

                    modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();
                    modelBuilder.Entity<TestUnit>().ToTable("TestUnits");
                }

                public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken =
                    default)
                {
                    IEnumerable<EntityEntry<Domain.UnitBase>> entries =
                        ChangeTracker.Entries<Domain.UnitBase>()
                            .Where(e => e.State is EntityState.Added or EntityState.Modified);

                    foreach (EntityEntry<Domain.UnitBase> entry in entries)
                    {
                        Domain.UnitBase unit = entry.Entity;
                        if (unit.ParentId.HasValue)
                        {
                            // At0C4@D46C 5CÄ @Câ4C„BCT;Dò 4C´O C0@Câ2CT@C=8 (C" -äÖV0÷'' f-æB @C 1CâBC 5D" 1D´AD$@Câ
                            Domain.UnitBase? parent = Set<Domain.UnitBase>().Find(unit.ParentId.Value);
                        }
                    }
                }
            }
        }
    }
}

```

```

        if (parent != null)
        {
            // 1. A@C 2C,,;Câ ÷6-F-öâ „"CT@CÄ8C00C'LC0KC' Cct5C²•
            if (parent.Kind == UnitKind.Position)
                throw new InvalidOperationException("Position node cannot have

children.");
        }

        // 2. A@C 2C,,;Câ FW 'FÖVçB „ CR <Cä6CTB D >CD5D 6C BDÂ ÷6-F-öâ•
        if (parent.Kind == UnitKind.Department && unit.Kind == UnitKind.Position)
            throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cfç >C JCT:D" :C B

'{parent.Kind}'.");
        }

        // 3. A@C 2C,,;Câ 8Ct>C'ODd8C, ... &öGV7F-öâö7F÷& vRöG& ç7 ÷'B =CR ACÄ5D,,8C$0DäBD O)
        if (parent.Kind is UnitKind.Production or UnitKind.Storage or

UnitKind.Transport)
        {
            if (unit.Kind != parent.Kind && unit.Kind != UnitKind.Position)
                throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cfç w·Væ-Bä¶-æ

'{parent.Kind}'.");
        }
    }
}

return await base.SaveChangesAsync(cancellationToken);
}

[Fact]
public async Task GetByKindAsync_Should_Return_Units_Matching_Complex_Mask()
{
    // Arrange
    var service = new TestUnitService(_factory);

    // B0GCT9C00 C$ECä4C,,B C" <C AC0C Storage
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Cell_01", Kind =

UnitKind.Storage, Type = UnitType.Cell });
    // B BC =Cä: C$ECä4C,,B C" <C AC0C Production
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Lathe_01", Kind =

UnitKind.Production, Type = UnitType.MachineTool });

    // Act
    List<Domain.UnitBase> storageUnits = await service.GetByKindAsync(UnitKind.Storage);

    // Assert
    storageUnits.Should().ContainSingle(x => x.Name == "Cell_01");
    storageUnits.Should().NotContain(x => x.Name == "Lathe_01");
}

[Fact]
public async Task GetByTypeAsync_Should_Handle_Specific_Flags()
{
    // Arrange
    var service = new TestUnitService(_factory);
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Crane_1", Kind =

UnitKind.Storage, Type = UnitType.Crane });
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Vehicle_1", Kind =

UnitKind.Transport, Type = UnitType.Vehicle });

    // Act
    List<Domain.UnitBase> result = await service.GetByTypeAsync(UnitType.Crane);

    // Assert
    result.Should().ContainSingle(x => x.Name == "Crane_1");
}

[Fact]
public async Task MoveAsync_Should_Throw_When_Moving_Into_Position()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var posId = Guid.NewGuid();
    var machineId = Guid.NewGuid();

```

```

        await service.AddAsync(new TestUnit { Id = posId, Name = "Terminal Point", Kind =
UnitKind.Position, Type = UnitType.Other });ð
        await service.AddAsync(new TestUnit { Id = machineId, Name = "Mobile Machine", Kind =
UnitKind.Production, Type = UnitType.MachineTool });ð
ð
        // Act & Assertð
        Func<Task> act = async () => await service.MoveAsync(machineId, posId);ð
ð
        await act.Should().ThrowAsync<InvalidOperationException>()ð
            .WithMessage("Position node cannot have children.");ð
    }ð
ð
    [Fact]ð
    public async Task MoveAsync_Should_Prevent_Circular_Dependencies()ð
    {ð
        // Arrangeð
        var service = new TestUnitService(_factory);ð
        var idA = Guid.NewGuid();ð
        var idB = Guid.NewGuid();ð
ð
        // AD>C 0C$;Dô5CÂ ¶-æB 8 Type, D$0C¢ :C : Cä=C, >C 0Ct0D$5C´LCÔK (required / init)ð
        await service.AddAsync(new TestUnitð
        {ð
            Id = idA,ð
            Name = "Node A",ð
            Kind = UnitKind.Department,ð
            Type = UnitType.Workshopð
        });ð
ð
        await service.AddAsync(new TestUnitð
        {ð
            Id = idB,ð
            Name = "Node B",ð
            ParentId = idA,ð
            Kind = UnitKind.Production,ð
            Type = UnitType.Sectionð
        });ð
ð
        // Act & Assertð
        Func<Task> act = async () => await service.MoveAsync(idA, idB);ð
ð
        await act.Should().ThrowAsync<InvalidOperationException>()ð
            .WithMessage("Bd8C¢;C„GCTAC¢0Dð 7C 2C„AC„<CäAD$L*");ð
    }ð
ð
    [Fact]ð
    public async Task GetFullTreeAsync_Should_Maintain_Hierarchy_And_References()ð
    {ð
        // Arrangeð
        var service = new TestUnitService(_factory);ð
        var rootId = Guid.NewGuid();ð
        var sectionId = Guid.NewGuid();ð
ð
        await service.AddAsync(new TestUnitð
        {ð
            Id = rootId,ð
            Name = "Dept",ð
            Kind = UnitKind.Department,ð
            Type = UnitType.Workshopð
        });ð
ð
        await service.AddAsync(new TestUnitð
        {ð
            Id = sectionId,ð
            Name = "Section",ð
            Kind = UnitKind.Production,ð
            Type = UnitType.Section,ð
            ParentId = rootIdð
        });ð
ð
        await service.AddAsync(new TestUnitð
        {ð
            Id = Guid.NewGuid(),ð
            Name = "Workstation",ð
            Kind = UnitKind.Production,ð
            Type = UnitType.Workstation,ð

```



```

        ParentId = sectionId
    });

    // Act
    Domain.UnitBase? tree = await service.GetFullTreeAsync(rootId);

    // Assert
    tree.Should().NotBeNull();
    tree!.Children.Should().HaveCount(1);
    tree.Children.First().Children.Should().HaveCount(1);
}

[Fact]
public async Task MoveAsync_Department_Should_Not_Contain_Position()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var deptId = Guid.NewGuid();
    var positionId = Guid.NewGuid();

    // 1. B >Ct4C 5CÂ CT?C @D$0CÄ5CÔB
    await service.AddAsync(new TestUnit
    {
        Id = deptId,
        Name = "Main Dept",
        Kind = UnitKind.Department,
        Type = UnitType.Workshop
    });

    // 2. B >Ct4C 5CÂ Cä7C,,FC,,N (D >C4;C ACÔ> D$2Cä5C' <C AC=5, DÔBCâ <Cä6CTB C KD$L Cell)
    await service.AddAsync(new TestUnit
    {
        Id = positionId,
        Name = "Work Point 1",
        Kind = UnitKind.Position,
        Type = UnitType.Cell
    });

    // Act
    // AôKD$0CT<D 0 Cô5D 5CÄ5D BC,,BDÂ ÷6-F-ôâ 2 Department
    Func<Task> act = async () => await service.MoveAsync(positionId, deptId);

    // Assert
    // Aä6C,,4C 5CÂ >D,,8C :D2 >D" Væ-D& 6T†-W& &6‡•G&-vvW-
    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage($"*Cø0D$5C4>D 8C, u ÷6-F-ôâr =CR <Cä6CTB C KD$L C$;Cä6CT= C" tFW 'FÖVçBrç""½
}

[Fact]
public async Task MoveAsync_Production_Should_Contain_Production_Subnode()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var parentProdId = Guid.NewGuid();
    var childProdId = Guid.NewGuid();

    await service.AddAsync(new TestUnit
    {
        Id = parentProdId,
        Name = "Production Area",
        Kind = UnitKind.Production,
        Type = UnitType.Workshop
    });

    await service.AddAsync(new TestUnit
    {
        Id = childProdId,
        Name = "Machine Node",
        Kind = UnitKind.Production,
        Type = UnitType.MachineTool
    });

    // Act
    Func<Task> act = async () => await service.MoveAsync(childProdId, parentProdId);

    // Assert

```

```

        // At4CTADÂ 8D :C´NDt5CÔ8Dò 1D´BDÂ =CR 4Cä;Cd=CâÂ BC : Cª0C¢ &öGV7F-öâ <Cä6CTB D >CD5D 6C BDÂ &öGV7
        await act.Should().NotThrowAsync();ð
    }ð
    Domain.UnitBase? updated = await service.GetByIdAsync(childProdId);ð
    updated!.ParentId.Should().Be(parentProdId);ð
}ð
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs">
using Microsoft.EntityFrameworkCore;ð
ð
namespace Promatis.Net.MES.Tests.UnitBase;ð
ð
// B$5D BCä2D´9 Cª>CÔBCT:D B C$=D4BD 8 Cò@CäAD$@C =D BC$0 C,,<CT= D$5D BCä2ð
public class AppDbContext : DbContextð
{ð
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }ð
    ð
    // A,,ACò>C´LctCCT< C 0Ct>C$KC´ BC,,?, DtBCä1D² BD 8C43CT@ CÄ>C2 4CT;C BDÂ 6WCÄVæ-D& 6Sâ,•
    public DbSet<Domain.UnitBase> Units => Set<Domain.UnitBase>();ð
    ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)ð
    {ð
        // 1. A00D BD 0C,,2C 5CÄ E B ,, : C" >D =Cä2CÔ>CÄ ?D >CT:D$5)ð
        modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();ð
    ð
        // 2. B4:C 7D´2C 5CÄÂ GD$> TestUnit – DÔBCâ =C AC´5CD=C,,:ð
        modelBuilder.Entity<TestUnit>().ToTable("TestUnits");ð
    }ð
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnit.cs">
namespace Promatis.Net.MES.Tests.UnitBase;ð
ð
public class TestUnit : MES.Domain.UnitBase { }
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs">
using Promatis.Net.MES.Domain;ð
ð
namespace Promatis.Net.MES.Tests.UnitBase;ð
ð
public class TestUnitValidator : UnitBaseValidator<TestUnit> { }
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs">
using FluentAssertions;ð
using FluentValidation.TestHelper;ð
using Microsoft.EntityFrameworkCore;ð
using Promatis.Net.Data;ð
using Promatis.Net.MES.Data;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Tests.UnitBase;ð
ð
public class UnitBaseArchitectureTestsð
{ð
    private readonly TestUnitValidator _validator = new();ð
    private readonly DbContextOptions<AppDbContext> _options;ð
    ð
    public UnitBaseArchitectureTests()ð
    {ð
        _options = new DbContextOptionsBuilder<AppDbContext>().ð
            .UseInMemoryDatabase(Guid.NewGuid().ToString())ð
            .Options;ð
    }ð
    ð
    private static EntityCancelEventArgs<MES.Domain.UnitBase> CreateArgs(MES.Domain.UnitBase
entity, DbContext context)ð
    {ð
        return new EntityCancelEventArgs<MES.Domain.UnitBase>(ð
            entity,ð
            EntityStateChangeEnum.Added,ð
            new List<PropertyChangeInfo>(),ð

```

```

        context);
    }
}

#region A$0C'8CD0Dd8D0 ,, C AC=8 C, 1C 7Cä2D'5 C0@C 2C,,C •

[Theory]
[InlineData(UnitKind.Storage, UnitType.Cell, true)] // B0GCT9C=0 C" !C=C 4CR r C-
[InlineData(UnitKind.Storage, UnitType.Workshop, false)] // Bd5DR 2 B :C'0CD5 - AäHC,,1C=0 (C 8D" =CR ACä2
[InlineData(UnitKind.Production, UnitType.Table, true)] // B BCä; C" D >C,,7C$>CDAD$2CR r C-
[InlineData(UnitKind.Position, UnitType.Other, true)] // A0@CäGCT5 C" Cä7C,,FC,,8 - Aä:D
[InlineData(UnitKind.Position, UnitType.Table, false)] // B BCä; C" Cä7C,,FC,,8 - AäHC,,1C=0 (D >C4;C AC0>
public void Validator_Should_Check_Bitwise_Compatibility(UnitKind kind, UnitType type, bool
isValid)
{
    var unit = new TestUnit { Kind = kind, Type = type, Name = "Test" };
    TestValidationResult<TestUnit>? result = _validator.TestValidate(unit);

    if (isValid)
    {
        result.ShouldNotHaveAnyValidationErrors();
    }
    else
    {
        result.ShouldHaveValidationErrorFor(x => x)
            .WithErrorMessage("B$8C0 =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
    }
}

#endregion

#region B$@C,,3C45D ,, D 0C$8C'0 C$;Cä6CT=C0>D BC, ¶-æB•

[Fact]
public async Task Trigger_Should_Allow_Root_Units()
{
    await using var context = new AppDbContext(_options);
    var root = new TestUnit { ParentId = null, Kind = UnitKind.Department, Type =
UnitType.Other };

    var trigger = new UnitBaseHierarchyTrigger();
    EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(root, context);

    await trigger.HandleAsync(args);

    args.Cancel.Should().BeFalse();
}

[Fact]
public async Task Trigger_Should_Reject_Position_In_Department()
{
    await using var context = new AppDbContext(_options);
    var parentId = Guid.NewGuid();

    context.Units.Add(new TestUnit { Id = parentId, Kind = UnitKind.Department, Type =
UnitType.Other, Name = "Dept" });
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestUnit { ParentId = parentId, Kind = UnitKind.Position, Type =
UnitType.Other, Name = "Pos" };
    var trigger = new UnitBaseHierarchyTrigger();
    EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);

    await trigger.HandleAsync(args);

    args.Cancel.Should().BeTrue();
    args.ErrorMessage.Should().Contain("C05 CÄ>Cd5D" 1D'BDÂ 2C'>Cd5C0 2 'Dept' (Department));
}

[Theory]
[InlineData(UnitKind.Production, UnitKind.Production, false)] // Bd5DR 2 Bd5DR r C-
[InlineData(UnitKind.Production, UnitKind.Transport, true)] // B$@C =D ?Cä@D" 2 Bd5DR r C ?D 5D"5C0> (
[InlineData(UnitKind.Storage, UnitKind.Position, false)] // A0>Ct8Dd8D0 2 B :C'0CB r C-
[InlineData(UnitKind.Position, UnitKind.Position, true)] // A0>Ct8Dd8D0 2 A0>Ct8Dd8Dâ r C ?D 5D"5C0
public async Task Trigger_Should_Enforce_Isolation(UnitKind pKind, UnitKind cKind, bool
shouldCancel)
{

```

```

        await using var context = new AppDbContext(_options);
        var parentId = Guid.NewGuid();

        // B >Ct4C 5CÄ @Cä4C,,BCT;Dò A Cò>CDECä4DôIC,,< D$8Cò>CÍ
        context.Units.Add(new TestUnit { Id = parentId, Kind = pKind, Type = GetFirstType(pKind),
Name = "P" });
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        var child = new TestUnit { ParentId = parentId, Kind = cKind, Type = GetFirstType(cKind),
Name = "C" };
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);
        await new UnitBaseHierarchyTrigger().HandleAsync(args);

        args.Cancel.Should().Be(shouldCancel, $"Cò>D$>CÄ DtBCä 2C´>Cd5Cò8CR ¶4¶-æGò 2 {pKind} CD>C´6Cò> Cò@C
    }
}
#endregion

private static UnitType GetFirstType(UnitKind kind)
    => Enum.GetValues<UnitType>().First(t => t != UnitType.None && ((int)kind & (int)t) != 0);
}
</file>

<file path="Promatis.Net.MES.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using MudBlazor
@using Promatis.Net.UI.Components.Workspace
@using Promatis.Net.UI.Components.EditDialog
@using Promatis.Net.UI.Components.Grid
@using Promatis.Net.UI.Components.Toolbar
@using Promatis.Net.UI.Components.Tree
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor">
<EditDialog Title="@Title"
    IsNew="@IsNew"
    Model="@Model"
    Validator="@Validator"
    OnSaveAction="@OnSaveAction">
    <Tabs>
        <DialogTab Title="AäACò>C$=Cä5" Icon="@Icons.Material.Filled.Info">
            <Content>
                <div class="d-flex flex-column gap-4 pt-2">
                    <MudTextField Label="Aä@C BCä>CR >C >Ct=C GCT=C,,5 (Aä>CB'"
                        @bind-Value="Model.Code"
                        For="@(() => Model.Code)"
                        Margin="Margin.Dense"
                        Variant="Variant.Outlined"
                        Required="true"
                        HelperText="Aô0Cò@C,,<CT@: CÄ<, Cä1/CÄ8CòÂ 2Â HD"" óí
                    <MudTextField Label="Aô>C´=Cä5 Cò0C,,<CT=Cä2C =C,,5"
                        @bind-Value="Model.Name"
                        For="@(() => Model.Name)"
                        Margin="Margin.Dense"
                        Variant="Variant.Outlined"
                        Required="true" />
                    <MudTextField Label="Aä?C,,AC =C,,5 / Aô@C,,<CTGC =C,,5"
                        @bind-Value="Model.Description"
                        For="@(() => Model.Description)"
                        Margin="Margin.Dense"
                        Variant="Variant.Outlined"
                        Lines="3" />
                </div>
            </Content>
        </DialogTab>
    </Tabs>
</EditDialog>
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor.cs">

```

```

using Microsoft.AspNetCore.Components;
using Promatis.Net.MES.Domain;
namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements.Card;
public partial class UnitOfMeasurementEditDialog : ComponentBase
{
    [Parameter] public required string Title { get; set; }
    [Parameter] public required bool IsNew { get; set; }
    [Parameter] public required UnitOfMeasurement Model { get; set; }
    [Parameter] public required FluentValidation.Validator Validator { get; set; }
    [Parameter] public required Func<Task> OnSaveAction { get; set; }
}
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor">
@page "/mes/mdm/units-of-measurement"
@using Promatis.Net.MES.Domain
@inject UnitOfMeasurementPageContext Context
<CascadingValue Value="@Context" IsFixed="true">
    <WorkspacePage>
        <TopContent>
            <UiToolbar TEntity="UnitOfMeasurement"
                ActionContext="@Context"
                OnCreateTriggered="@Context.OnCreateActionAsync"
                OnEditTriggered="@Context.OnUpdateActionAsync"
                OnDeleteTriggered="@Context.OnDeleteActionAsync" />
        </TopContent>
        <BodyContent>
            @* B$ A A,&A $ C =C0KCR ?D >C8CDKC$0DäBD O C0C0@D0<D4N C,,7 C=C0BCT:D BC ?Cä BD 5C >C$0C08Dä
            <GridPage TEntity="UnitOfMeasurement">
                <ColumnsContent>
                    <PropertyColumn T="UnitOfMeasurement" TProperty="string" Property="x => x.Code"
                        Title="A@C BC>CR >C >Ct=C GCT=C,,5" HeaderStyle="width: 150px;" Class="font-monospace" />
                    <PropertyColumn T="UnitOfMeasurement" TProperty="string" Property="x => x.Name"
                        Title="A0>C´=Cä5 C00C,,<CT=Cä2C =C,,5" HeaderStyle="width: 300px;" />
                    <PropertyColumn T="UnitOfMeasurement" TProperty="string" Property="x =>
                        x.Description" Title="Aä?C,,AC =C,,5 / A0@C,,<CTGC =C,,5" />
                </ColumnsContent>
            </GridPage>
        </BodyContent>
    </WorkspacePage>
</CascadingValue>
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPageContext.cs">
using FluentValidation;
using MudBlazor;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Service;
using Promatis.Net.MES.UI.Pages.UnitOfMeasurements.Card;
using Promatis.Net.UI.Components.Grid;
namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements;
public class UnitOfMeasurementPageContext : GridActionContext<UnitOfMeasurement>
{
    private readonly IUnitOfMeasurementService _uomService;
    private readonly IDialogService _dialogService;
    private readonly IValidator<UnitOfMeasurement> _globalValidator;

    public UnitOfMeasurementPageContext(
        IUnitOfMeasurementService uomService,
        IDialogService dialogService,
        IValidator<UnitOfMeasurement> globalValidator) : base()
    {
        _uomService = uomService ?? throw new ArgumentNullException(nameof(uomService));
        _dialogService = dialogService ?? throw new ArgumentNullException(nameof(dialogService));
        _globalValidator = globalValidator ?? throw new
        ArgumentNullException(nameof(globalValidator));
    }
}

```

```

    PageTitle = "B ?D 0C$>Dt=C,,: CT4C,,=C,,F C,,7CÄ5D 5CÖ8Dò#½
}
}
/// <summary>
/// Bt8D BD´9 Cö@C,,:C´0CD=Cä9 CÄ5D$>CB 7C 3D CCT:C, 4C =CÖKDR 8Cr !B4 AB 2 Ää B2Ô:DÖH CD;Dò ?D 8C$0Ct:C,
/// </summary>
public async Task<GridData<UnitOfMeasurement>> GetUnitsOfMeasurementAsync()
{
    List<UnitOfMeasurement> data = await _uomService.GetAllAsync();
    return new GridData<UnitOfMeasurement>
    {
        Items = data,
        TotalItems = data.Count
    };
}

// =====
// A AÄ AÖ B² A AT A, AÖ!B$ B4 AT B$ A" „5%TB•
// =====

public async Task OnCreateActionAsync()
{
    var newUom = new UnitOfMeasurement();
    await OpenEditDialogAsync(newUom, "AD>C 0C$;CT=C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0", isNew: true);
}

public async Task OnUpdateActionAsync(UnitOfMeasurement row)
{
    if (row == null) return;
    var targetClone = CloneEntity(row);
    await OpenEditDialogAsync(targetClone, $"B 5CD0C=BC,,@Cä2C =C,,5: {row.Code}", isNew: false);
}

public async Task OnDeleteActionAsync(UnitOfMeasurement row)
{
    if (row == null) return;

    var parameters = new DialogParameters { ["ContentText"] = $"B44C ;C,,BDÄ 5CD8CÖ8DdC C,,7CÄ5D 5CÖ8Dò
'{row.Name}' ({row.Code})?" };
    var options = new DialogOptions { CloseOnEscapeKey = true };

    var dialog = await _dialogService.ShowAsync<MudMessageBox>("B44C ;CT=C,,5 Cä1D=5C=BC "Ä & ÖWFW'2Ä ÷
    var result = await dialog.Result;

    if (result is not null && !result.Canceled)
    {
        await _uomService.DeleteAsync(row.Id);
        RequestRefresh();
    }
}

private async Task OpenEditDialogAsync(UnitOfMeasurement model, string title, bool isNew)
{
    var parameters = new DialogParameters
    {
        ["Title"] = title,
        ["IsNew"] = isNew,
        ["Model"] = model,
        ["Validator"] = _globalValidator,
        ["OnSaveAction"] = async () =>
        {
            if (isNew)
                await _uomService.AddAsync(model);
            else
                await _uomService.UpdateAsync(model);

            RequestRefresh();
        }
    };

    var options = new DialogOptions { CloseButton = true, MaxWidth = MaxWidth.Small, FullWidth
= true };
    await _dialogService.ShowAsync<UnitOfMeasurementEditDialog>(title, parameters, options);
}
}

```

</file>

<file path="Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj">

<Project Sdk="Microsoft.NET.Sdk.Razor">

ð

<PropertyGroup>

<TargetFramework>net10.0</TargetFramework>

<Nullable>enable</Nullable>

<ImplicitUsings>enable</ImplicitUsings>

</PropertyGroup>

ð

<ItemGroup>

<Compile Remove="UnitBase*" />

<Content Remove="UnitBase*" />

<EmbeddedResource Remove="UnitBase*" />

<None Remove="UnitBase*" />

</ItemGroup>

ð

ð

<ItemGroup>

<SupportedPlatform Include="browser" />

</ItemGroup>

ð

<ItemGroup>

<PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />

<PackageReference Include="MudBlazor" Version="9.4.0" />

</ItemGroup>

ð

<ItemGroup>

<ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" /

>

<ProjectReference Include="..\..\Core\Promatis.Net.UI\Promatis.Net.UI.csproj" />

<ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />

</ItemGroup>

ð

</Project>

</file>

<file path="Promatis.Net.MES.UI/UiModule.cs">

using MudBlazor;

using Promatis.Net.UI;

ð

namespace Promatis.Net.MES.UI;

ð

public class UiModule : IUiModule

{

public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

ð

public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()

{

return new List<(string Title, string Href, string Icon, string? Group)>

{

// Að;CäAC8C' ACôC 2CäGC08Cø BCTECô>C'>C48Dt5D :C,,E C00D 0CÄ5D\$@Cä2

("AT4C,,=C,,FD² 8Ct<CT@CT=C,,0", "/mes/mdm/units-of-measurement", Icons.Material.Filled.SettingsInput

});

};

}

</file>

<file path="Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js">

// This is a JavaScript module that is loaded on demand. It can export any number of

// functions, and may import other JavaScript modules if required.

ð

export function showPrompt(message) {

return prompt(message, 'Type anything here');

}

</file>

</files>